Unit IV

4. Software Testing

4.1 Definition
4.2 Verification and Validation
4.3 Black box Testing
4.4 White-Box Testing
4.5 Unit Testing

4.1 Definition:

Software Testing – Definition:

Software Testing is the process of evaluating a software application to ensure that it works correctly, meets user requirements, and is free from defects. It involves running the software under controlled conditions to identify errors, gaps, or missing requirements.

Software Testing checks **both the functionality and performance** of a system and ensures that the final product is **reliable, secure, and of good quality** before delivery.

Key Points:

1. **Verification and Validation:**
   Testing verifies whether the software is built correctly and validates whether it meets user needs.

2. **Defect Identification:**
   It helps find bugs, logical errors, wrong calculations, and missing functionalities.

3. **Quality Assurance:**
   Ensures the software is stable, user-friendly, and performs as expected.

4. **Risk Reduction:**
   Detecting defects early reduces risk of failure after deployment.

5. **Improves Reliability:**
   A well-tested system increases customer trust and ensures dependable performance.

4.2 Verification and Validation:

Verification and Validation (V&V) are two important activities in software engineering used to ensure the quality of a software product.

Verification:

Verification is the process of **checking whether the software is being built correctly**.
It focuses on **design, documents, and processes** rather than the final product.

Key Points:

It ensures that each phase of development follows the specified requirements.
Performed through reviews, inspections, walkthroughs, and design analysis.
Answers the question: **"Are we building the product right?"**

Example: Checking if the design document correctly follows the user requirements.

Validation:

Validation is the process of **checking whether the correct software has been built**.
It ensures that the final product **meets the user's actual needs and expectations**.

Key Points:

Focuses on testing the finished product.
Performed through functional testing, system testing, user acceptance testing (UAT).
Answers the question: **"Are we building the right product?"**

Example: Running the software to confirm that it performs customer-required functions.

Conclusion:

Verification ensures that development is done **correctly**, while validation ensures that the **right product** has been developed. Together, V&V improve the **accuracy, safety, reliability, and quality** of the software.

4.3 Black box Testing:

Black Box Testing is a software testing technique in which the tester evaluates the functionality of a system **without looking at the internal code, structure, or logic**. The software is treated like a "black box," and only the **inputs and outputs** are examined.

In this method, the tester checks whether the software works according to the **requirements and specifications**. The internal program logic is not considered.

Key Features:

1. **Focus on functionality –** Tests what the system does, not how it works internally.
2. **No knowledge of code required –** Suitable for testers who only know the requirements.
3. **Based on user requirements –** Ensures the software meets customer expectations.
4. **Input–Output based testing –** Different inputs are given to observe outputs.
5. **Performed at different levels –** Used in system testing, acceptance testing, and functional testing.

Common Techniques:

Equivalence Partitioning (dividing inputs into valid/invalid classes)
Boundary Value Analysis (testing at the edges of input ranges)
Decision Table Testing
State Transition Testing

Example:

For a login page:

Valid username & password → Should login
Invalid password → Should show error
Empty fields → Should show "Enter details"

The tester does not check the code that verifies passwords.

Conclusion:

Black Box Testing ensures that the software **functions correctly**, satisfies user requirements, and behaves as expected for all input conditions, making it essential for delivering high-quality software.

4.4 White-Box Testing:

White-Box Testing (also called Structural Testing or Glass-Box Testing) is a software testing technique in which the tester examines the **internal structure, code, and logic** of the program. The tester must have knowledge of programming and the system's internal implementation.

In this method, test cases are designed based on **control flow**, **data flow**, **loops**, **conditions**, and **program structure** to ensure the code works correctly at every level.

Key Features:

1. **Internal logic tested –** Every path, branch, and condition inside the code is verified.
2. **Requires programming knowledge –** Testers must understand code and algorithms.
3. **Covers hidden errors –** Finds logical errors, incorrect assumptions, and security issues.
4. **Code-based test design –** Test cases are derived from program structure.
5. **Used in early stages –** Mostly done at unit testing and integration testing levels.

Common Techniques:

Statement Coverage – Every line of code is executed at least once.
Branch/Decision Coverage – Each decision (IF, ELSE) is tested.
Path Coverage – All possible execution paths are verified.
Loop Testing – Ensures loops work correctly (0, 1, and multiple iterations).

Example:

Testing a function that calculates the largest of three numbers:

The tester checks all branches:

If A > B
If B > C
If C is largest
Handling equal values

The tester evaluates the actual code to confirm logic accuracy.

Conclusion:

White-Box Testing ensures that the **internal code is correct, efficient, secure, and error-free**. It helps identify logical mistakes that may not be visible through functional (black-box) testing alone.

4.5 Unit Testing:

Unit Testing is a software testing method in which individual components or **smallest testable units** of a program (such as functions, methods, or modules) are tested independently. The objective is to verify that each unit works correctly **in isolation** before combining them into a larger system.

Unit Testing is usually performed by **developers** during the coding phase and helps identify errors early in

the development cycle.

Key Features:

1. **Tests the smallest parts** of a system, such as a single function or procedure.
2. **Performed by programmers** since it requires code-level understanding.
3. **Ensures correctness of logic** within each unit.
4. **Helps detect errors early**, reducing fixing cost later.
5. **Improves code quality** through better structure and fewer bugs.

Advantages:

Easy to locate and fix errors since each unit is tested separately.
Supports code reusability by encouraging modular design.
Makes debugging simple and faster.
Reduces chances of defects during integration and system testing.

Example:

For a function that calculates the sum of two numbers, the developer tests:

sum(2, 3) → 5
sum(-1, 6) → 5
sum(0, 0) → 0

Each case checks only that one function works correctly.

Conclusion:

Unit Testing ensures that every building block of the software is **reliable and error-free**, leading to a stable and high-quality final system.